CMSC 25750: Large Language Models
Homework 3: Frontier Lab Interview Prep

Instructor: Chenhao Tan

Due: May 3, 2026, 11:59 pm

This homework covers two topics that follow those in technical interviews at frontier AI labs (OpenAI, Anthropic, Google DeepMind, Meta FAIR, etc.): **distributed training** and **attention residuals**. Each question involves both implementation and analytical reasoning. No GPU required—everything can be run on CPU.

Getting Started We host all the code snippets and relevant files on this GitHub repo. Pull the assignment repository to get all necessary files:

```
git pull git@github.com:uchicago-llm-course/spring2026-assignments.git
```

Submission Export your `hw3.ipynb` to a PDF file, and then upload to Gradescope with all cell outputs visible. This PDF is a read-only problem sheet; do **not** submit it.

Repository layout

```
HW3/
|-- hw3.ipynb              (submit this; run all cells, write answers here)
|-- hw3.pdf                (this problem sheet, read only)
|-- src/
|   |-- distributed.py     (implement Question 1)
|   |-- attn_residuals.py  (implement Question 2)
|-- tests/
|   |-- test_distributed.py(autograder; do not modify)
|   |-- test_attn_res.py   (autograder; do not modify)
|-- pyproject.toml
```

General Instructions See `README.md` for environment setup. Work one question at a time:

1. Implement the coding problems in `src/distributed.py` and `src/attn_residuals.py` (look for `YOUR CODE HERE` markers).
2. Run the autograder for the completed question:

   ```
   pytest tests/test_distributed.py -v   # after Question 1
   pytest tests/test_attn_res.py -v      # after Question 2
   ```

3. Open `hw3.ipynb`, run the autograder cell for that question, and write your answers in the designated markdown cells.
4. When both questions are done, export `hw3.ipynb` as PDF and submit to Gradescope.

Acknowledgements
This assignment draws on ideas from Stanford CS336 (Hashimoto & Liang), MLE-LeetCode (Jonnes Lin), and LLM Inference Interview Questions (Eitan Turok).

Question 1: Distributed Training (50 pts)

Training large language models requires spreading work across multiple GPUs. **Distributed Data Parallelism (DDP)** (PyTorch DDP) is the most widely used strategy: each GPU holds a full copy of the model and processes a different mini-batch; after the backward pass, gradients are **allreduced** (averaged) so that all replicas apply the same optimizer step and stay in sync.

Ring allreduce. The simplest way to average gradients is to send them all to one GPU, average there, and broadcast the result back. The problem is if the model has P parameters at b bytes each (e.g., $b = 2$ for FP16), that coordinator must receive $P \cdot b$ bytes from *every* other GPU, making it a bandwidth bottleneck that worsens as you add GPUs.

Gibiansky, 2017 eliminates this bottleneck by arranging N GPUs in a logical ring where each GPU only communicates with its two neighbours. Each GPU splits its gradient tensor into N equal chunks and the algorithm proceeds in two phases:

1. **Scatter-reduce** ($N-1$ rounds). Consider $N = 4$ GPUs, each with chunks $[c_0, c_1, c_2, c_3]$. Let $g_i[j]$ denote GPU i's local value of chunk j. In round 0, GPU i sends chunk c_i (its "own" chunk) to GPU $(i+1)$. In every subsequent round, each GPU forwards whichever chunk it *received* in the previous round (now with the partial sum folded in). The receiver always adds the incoming values to its local copy:

	GPU 0 sends	GPU 1 sends	GPU 2 sends	GPU 3 sends
Round 0	$c_0 \to$ GPU 1	$c_1 \to$ GPU 2	$c_2 \to$ GPU 3	$c_3 \to$ GPU 0
Round 1	$c_3 \to$ GPU 1	$c_0 \to$ GPU 2	$c_1 \to$ GPU 3	$c_2 \to$ GPU 0
Round 2	$c_2 \to$ GPU 1	$c_3 \to$ GPU 2	$c_0 \to$ GPU 3	$c_1 \to$ GPU 0

   Follow chunk c_1: GPU 1 sends it in round 0, carrying $g_1[1]$. GPU 2 adds its own value, giving $g_1[1]+g_2[1]$, then sends it in round 1. GPU 3 adds again $(g_1[1]+g_2[1]+g_3[1])$ and sends in round 2. GPU 0 adds last: $g_0[1]+g_1[1]+g_2[1]+g_3[1]$ — the full sum. All four chunks snowball in parallel, each finishing on a different GPU after 3 rounds.

2. **Allgather** ($N-1$ rounds). Each GPU now has one fully-reduced chunk. Using the same ring pattern but *copying* instead of adding, each GPU broadcasts its chunk so that after $N-1$ rounds every GPU has all N reduced chunks — the complete averaged gradient.

Communication cost. In each phase, every GPU sends $(N-1)$ chunks of size $P \cdot b/N$ bytes, giving a total per-GPU transfer of:

$$2 \cdot \frac{N-1}{N} \cdot P \cdot b \text{ bytes}$$

As N grows, this approaches $2Pb$—the cost per GPU is bounded by twice the gradient size and does *not* grow with the number of GPUs. This is what makes ring allreduce *bandwidth-optimal*: adding more GPUs does not increase each GPU's communication burden. For a visual walkthrough with step-by-step diagrams see Gibiansky's blog post.

Coding Problems

1.1: Implement `ring_allreduce` **(16 pts)**

In `src/distributed.py`, implement `ring_allreduce(tensors)`.
`tensors` is a list of N tensors (one per simulated GPU), all the same shape, representing each GPU's local gradient. Simulate the ring allreduce algorithm **without** using any `torch.distributed` primitives:

1. **Chunk**: split each tensor into N equal-sized chunks.
2. **Scatter-reduce**: for $N-1$ rounds, GPU i sends its current chunk to GPU $(i+1) \bmod N$, which adds the received values to its own copy. In round 0 GPU i sends chunk i; in each subsequent round it forwards the chunk it just received.
3. **Allgather**: for $N-1$ rounds, each GPU sends its fully reduced chunk to the next GPU, which copies it into the correct position.
4. **Average**: divide all chunks by N.

Return a list of N tensors, each containing the averaged gradient. All output tensors must be identical (within `atol=1e-6`).

1.2: Implement `simulate_ddp_step` **(13 pts)**

In `src/distributed.py`, implement `simulate_ddp_step(model_fn, optimizer_fn, data_shards, loss_fn)`.
Simulate one DDP training step across N GPUs without using `torch.distributed`:

1. `model_fn()` returns a freshly initialized model. `optimizer_fn(params)` returns an optimizer. Create N independent (model, optimizer) pairs, all starting from the *same* initial weights (use the same random seed or copy the state dict).
2. `data_shards` is a list of N `(input, target)` tuples, one per GPU. Run a forward + backward pass on each model with its own shard.
3. Collect the gradient tensors (flattened) from all N models and call your `ring_allreduce` to average them.
4. Load the averaged gradients back into each model and call `optimizer.step()`.

Return the list of N models. The autograder checks two things:

- All N models have *identical* parameters after the step.
- Those parameters match a reference computed by naively averaging the gradients (i.e. summing all N gradient tensors and dividing by N, without ring allreduce).

1.3 Analysis

(a). **[9 pts] Communication cost.** You have $N = 8$ A100 GPUs connected by NVLink at 300 GB/s (bidirectional per link), training a 7B-parameter model in FP16 ($b = 2$ bytes/-param).

(i) How many bytes does each GPU send during ring allreduce?

(ii) Estimate the allreduce's wall-clock time, i.e., the elapsed real-world time, assuming the operation is *bandwidth-bound* on a fully-utilized NVLink ring (ignore per-message latency

and setup overhead, and treat the ring as a pipeline whose throughput is set by its slowest link).

(iii) Suppose the forward + backward pass takes 1.5 seconds and communication runs *serially* after compute (i.e., no overlap between the two). What fraction of the total step time (compute + communication) is spent on communication?

Deliverable: Numerical calculations.

(b). **[12 pts] Scaling and design tradeoffs.**

(i) Now suppose the 8 GPUs are spread across 2 nodes connected by InfiniBand at 50 GB/s (instead of NVLink at 300 GB/s). Recompute the allreduce time. What fraction of step time is communication now? What does this tell you about the difference between *intra-node* and *inter-node* training?

(ii) DDP overlaps communication with the backward pass by firing off allreduce for each *gradient bucket* (a group of parameters bundled together for one allreduce, ∼25 MB by default) as soon as all gradients in that bucket are ready. Explain why this helps and what limits the improvement (i.e., when does overlap fail to fully hide communication?).

Deliverable: Calculations with 2–3 sentences of explanation per part.

Question 2: Attention Residuals (50 pts)

Residual connections make deep Transformers trainable, but the way modern LLMs combine them with layer normalization has a subtle cost. This section covers a recent fix (Kimi Team, 2026) that replaces the fixed residual sum with a learned, attention-based mixture over previous layer outputs.

PostNorm vs. PreNorm. There are two conventions for where to apply LayerNorm in a Transformer block:

$$\begin{aligned} \text{PostNorm (original Transformer):} \quad & \boldsymbol{h}_{l+1} = \mathrm{LN}\big(\boldsymbol{h}_l + f_l(\boldsymbol{h}_l)\big) \\ \text{PreNorm (modern LLMs):} \quad & \boldsymbol{h}_{l+1} = \boldsymbol{h}_l + f_l\big(\mathrm{LN}(\boldsymbol{h}_l)\big) \end{aligned}$$

PreNorm moves the LN *inside* the residual branch, leaving a clean identity path from input to output. This is what gives deep LLMs (GPT, LLaMA, ...) stable gradients—and is also the source of the problem AttnRes attacks. (Modern LLMs actually use **RMSNorm**, a cheaper variant that omits mean subtraction; treat it as LayerNorm for this section.)

PreNorm dilution. Because the residual stream is never normalized, layer contributions accumulate unchecked:

$$\boldsymbol{h}_l \;=\; \boldsymbol{h}_1 \;+\; \sum_{i=1}^{l-1} f_i\big(\mathrm{LN}(\boldsymbol{h}_i)\big).$$

Each f_i sees a normalized input and standard init preserves scale through the sublayer, so $\|f_i(\mathrm{LN}(\boldsymbol{h}_i))\| \approx c$ is roughly constant in i. If the updates are roughly orthogonal across layers, the running residual stream has norm $\sim c\sqrt{l}$, so any one layer's contribution is only a $\sim 1/\sqrt{l}$ fraction of the total, a phenomenon known as **PreNorm dilution**. A second limitation is that a layer cannot *selectively* read from a specific earlier layer; $\boldsymbol{h}_l$ is a single fixed-weight bag-sum.

Attention Residuals (AttnRes) fixes both: each layer l computes its input as a softmax-attention mixture of all preceding layer outputs. The mechanism is structurally identical to softmax attention but operates over the **depth** dimension instead of the sequence dimension.

Formulation. Let $\boldsymbol{v}_0 = \boldsymbol{h}_1$ (the token embedding) and $\boldsymbol{v}_i = f_i(\mathrm{LN}(\boldsymbol{h}_i))$ for $i \in \{1, \ldots, l-1\}$ be the output of layer i (i.e., the PreNorm sublayer output, with its usual input LayerNorm applied inside). AttnRes introduces a layer-specific learnable **pseudo-query** $\boldsymbol{w}_l \in \mathbb{R}^d$ and uses keys equal to the values ($\boldsymbol{k}_i = \boldsymbol{v}_i$). The depth-wise attention weights are

$$\alpha_{i\to l} \;=\; \frac{\exp\big(\boldsymbol{w}_l^\top \mathrm{RMSNorm}(\boldsymbol{k}_i)\big)}{\sum_{j=0}^{l-1} \exp\big(\boldsymbol{w}_l^\top \mathrm{RMSNorm}(\boldsymbol{k}_j)\big)},$$

and the input to layer l is

$$\boldsymbol{h}_l \;=\; \sum_{i=0}^{l-1} \alpha_{i\to l}\, \boldsymbol{v}_i.$$

The RMSNorm on the keys is essential because raw layer outputs $\boldsymbol{k}_i$ have wildly different magnitudes across depth; without it the softmax would just pick whichever layer has the largest norm rather than the one most aligned with $\boldsymbol{w}_l$. Note also that the attention weights are computed **per token** (each token has its own keys), but $\boldsymbol{w}_l$ is a single d-vector shared across tokens.

Setting $\alpha_{l-1\to l} = 1$ recovers the standard residual; setting $\alpha_{i\to l} = 1/l$ recovers uniform accumulation. AttnRes lets the model *learn* which earlier layers to read from on a per-token basis, eliminating magnitude growth and giving every later layer a direct path to every earlier one.

Block AttnRes. Storing all L previous layer outputs costs $O(Ld)$ memory per token, which is expensive for deep models and pipeline-parallel training. **Block AttnRes** partitions the L layers into N blocks of size $S = L/N$. Within a block, layer outputs are summed normally (standard residual). *Across* blocks, AttnRes attends only over the N block-level summaries. This reduces memory from $O(Ld)$ to $O(Nd)$ and depth-wise attention compute from $O(L^2d)$ to $O(N^2d)$ while preserving most of the gain (empirically, $N \approx 8$ suffices).

Coding Problems

2.1: Implement `prenorm_residual_layer` (8 pts)

In `src/attn_residuals.py`, implement `prenorm_residual_layer(h, f, norm)`, which applies a single PreNorm residual update

$$\boldsymbol{h} \;\rightarrow\; \boldsymbol{h} + f(\mathrm{norm}(\boldsymbol{h})),$$

where `f` is an `nn.Module` (an attention or MLP block) and `norm` is an RMSNorm. This is the baseline that the AttnRes variants must reduce to when only a single previous layer output is available.

2.2: Implement `full_attn_res` (16 pts)

In `src/attn_residuals.py`, implement `full_attn_res(prev_outputs, query, norm)`, where:

- `prev_outputs` is a list of l tensors, each of shape `(B, T, d)`, holding $\boldsymbol{v}_0, \ldots, \boldsymbol{v}_{l-1}$.
- `query` is the layer-specific learnable pseudo-query $\boldsymbol{w}_l$, shape `(d,)`.
- `norm` is an RMSNorm module applied to the keys.

Compute the depth-wise softmax attention weights $\alpha_{i\rightarrow l}$ **per token** and return $\boldsymbol{h}_l$ of shape `(B, T, d)`.

Implement this without an explicit Python-level loop over the l previous layers—stack the previous outputs along a new "depth" axis and use a single `einsum` + `softmax`.

Hint: think carefully about what dimension the softmax is over. The pseudo-query is shared across tokens, but the keys (and therefore the α's) are not.

2.3: Implement `block_attn_res` (6 pts)

In `src/attn_residuals.py`, implement `block_attn_res(block_summaries, partial_block, query, proj, norm)` following Eq. (6) of the paper:

- `block_summaries` is a list of $N-1$ completed block summaries $\boldsymbol{b}_0, \ldots, \boldsymbol{b}_{N-2}$, each of shape `(B, T, d)`.
- `partial_block` is the running intra-block sum $\boldsymbol{b}_n^{i-1}$ for the current (in-progress) block.
- `proj` is a linear projection applied to the stacked keys before normalization.

Stack the keys $[\boldsymbol{b}_0, \ldots, \boldsymbol{b}_{N-2}, \boldsymbol{b}_n^{i-1}]$, project them with `proj`, apply `norm`, and compute softmax attention with `query`. Return the aggregated output of shape `(B, T, d)`.

2.4 Analysis

(a). **[8 pts] Cost of Full vs. Block AttnRes.** Consider a model with L layers, hidden dim d, sequence length T, batch size B, and FP16 activations (2 bytes per element).

(i) How many bytes of *additional* activation memory must be held during the forward pass for Full AttnRes, beyond what standard residual connections require? What about Block AttnRes with N blocks? Be precise about which tensors must be kept live and why.

(ii) How many extra FLOPs per token does the depth-wise attention add at the deepest layer for Full AttnRes versus Block AttnRes? Plug in $L = 64$, $N = 8$, $T = 4096$, $d = 4096$ and compare both numbers to the per-layer self-attention cost $O(Td^2 + T^2d)$. Why is the AttnRes overhead negligible at this scale?

Deliverable: Calculations with 2–3 sentences of explanation per part.

(b). **[12 pts] PreNorm dilution.** Throughout this problem, write $f_i := f_i(\mathrm{LN}(\boldsymbol{h}_i))$ as shorthand for the PreNorm sublayer output. Recall that a standard PreNorm Transformer produces $\boldsymbol{h}_l = \boldsymbol{h}_1 + \sum_{i=1}^{l-1} f_i$, whereas AttnRes produces $\boldsymbol{h}_l = \sum_{i=0}^{l-1} \alpha_{i\to l}\, \boldsymbol{v}_i$ with $\alpha_{i\to l} \geq 0$ and $\sum_i \alpha_{i\to l} = 1$.

(i) **Norm growth under standard PreNorm.** We will show that the residual stream's norm grows like $\sqrt{l}$, so a single layer's relative contribution shrinks like $1/\sqrt{l}$. Assume the layer outputs are approximately orthogonal in high dimension: $\mathbb{E}\langle f_i, f_j\rangle \approx 0$ for $i \neq j$ and that each $\mathbb{E}\|f_i\|^2 = c^2$ for some constant $c > 0$.

(a) **Expand.** Using bilinearity of the inner product ($\|\boldsymbol{u}\|^2 = \langle \boldsymbol{u}, \boldsymbol{u}\rangle$), expand

$$\Big\|\boldsymbol{h}_1 + \sum_{i=1}^{l-1} f_i\Big\|^2$$

into four term groups: $\|\boldsymbol{h}_1\|^2$, the $\boldsymbol{h}_1$–f_i cross terms, the diagonal $\|f_i\|^2$ terms, and the f_i–f_j cross terms ($i \neq j$).

(b) **Take expectations.** Apply the three assumptions above term-by-term to show

$$\mathbb{E}\|\boldsymbol{h}_l\|^2 \;\approx\; \|\boldsymbol{h}_1\|^2 + (l-1)\, c^2.$$

(c) **Conclude.** Since $\|\boldsymbol{h}_l\| \sim c\sqrt{l}$ but any single layer's contribution has norm $\sim c$, the relative contribution $\|f_i\|/\|\boldsymbol{h}_l\|$ shrinks like $1/\sqrt{l}$. In one sentence, explain why this is undesirable: how does it make deep models harder to train?

(ii) **Bounded norm under AttnRes.** Show that AttnRes, whose mixing weights satisfy $\sum_i \alpha_{i\to l} = 1$ and $\alpha_{i\to l} \geq 0$, keeps $\|\boldsymbol{h}_l\|$ bounded by $\max_i \|\boldsymbol{v}_i\|$ independent of depth. In one short sentence: where does the boundedness come from structurally?

Deliverable: A few lines of math per sub-part, plus the two short explanations requested in (i)(c) and (ii).